

Modular Monoliths Architecture

Modular Monoliths Architecture: When to Use, Benefits and Challenges

Siamo al **secondo passo** del percorso architetturale. Il problema ereditato dalla Lezione 2: *gestire ed evolvere il monolite è diventato complesso*.

Definizione

L'architettura monolitica modulare è un approccio in cui l'applicazione viene sviluppata e distribuita come un'unica unità coesa, ma è internamente suddivisa in moduli ben definiti e a basso accoppiamento.

I punti costitutivi:

1. **Si costruisce e si distribuisce comunque una singola applicazione**, esattamente come nel monolite tradizionale. Il deployment non cambia.
2. Internamente il codice è **suddiviso in moduli indipendenti**, uno per ogni capacità di business richiesta.
3. **Ogni modulo incapsula le proprie funzionalità** e le proprie operazioni interne, che restano **nascoste agli altri moduli** (forte incapsulamento).
4. **I moduli rappresentano un bounded context** del dominio applicativo: dentro un modulo si raggruppano le funzionalità di quel contesto.
5. **La comunicazione tra moduli avviene tramite interfacce chiare ed esplicite**. Un modulo comunica solo con quelli che gli forniscono il servizio specifico di cui ha bisogno.

Definizione: Bounded Context — concetto del Domain-Driven Design. È un confine esplicito entro il quale un modello di dominio è valido e coerente, con il proprio linguaggio. Nell'e-commerce: *Catalog, Basket, Ordering* sono bounded context diversi — la parola "Product" significa cose leggermente diverse in ciascuno.

L'obiettivo dichiarato (la frase chiave della lezione)

*"Ottenere i vantaggi di sviluppo e manutenzione di un'architettura modulare **senza ereditare immediatamente la complessità operativa di un'architettura distribuita a microservizi**."*

Questo è il senso profondo del pattern: **prendere i benefici di organizzazione del codice dei microservizi senza pagarne il prezzo operativo**.

Quando usarla — i 4 scenari

1. Il monolite esistente è diventato troppo complesso

Esattamente la nostra situazione. Se il monolite manca di struttura interna, diventa un Big Ball of Mud. Il modular monolith lo **scompone in parti gestibili e allineate al dominio**.

2. Migliorare autonomia del team e sviluppo parallelo

Con più team che lavorano su funzionalità di business diverse, ciascuno può **sviluppare, testare e persino refactorizzare il proprio modulo minimizzando il rischio per gli altri** — a condizione che rispettino le interfacce definite.

3. Servono confini chiari senza la complessità distribuita

Si ottengono molti benefici di organizzazione e manutenibilità dei microservizi **senza sostenere immediatamente i costi di gestione dei sistemi distribuiti**.

4. Preparare una transizione graduale ai microservizi ★

Questo è il motivo strategicamente più importante:

*"Un modular monolith ben progettato può semplificare notevolmente la futura transizione ai microservizi. Quando un modulo diventa candidato all'estrazione in un servizio proprio, **confini chiari e interfacce ben definite rendono il processo molto più semplice**. Il modulo funziona già come un microservizio all'interno del monolite."*

I 6 vantaggi

Vantaggio	Dettaglio
Codice organizzato	La codebase diventa molto più facile da navigare e analizzare quando è suddivisa in moduli chiari e focalizzati sul dominio
Manutenibilità e testabilità	È più facile e sicuro modificare o aggiungere funzionalità dentro un modulo senza impattare involontariamente altre parti
Autonomia dei team	Team diversi possono possedere moduli specifici e operare in modo più indipendente
Deployment semplice	Si distribuisce comunque un singolo artefatto , molto meno complesso che gestire N deployment distribuiti
Nessun overhead distribuito ★	Si evitano latenza di rete, protocolli inter-servizio, service discovery e distributed tracing che accompagnano i microservizi
Evoluzione futura facilitata	I confini già definiti rendono semplice il processo di estrazione di un modulo verso un microservizio

Esempio della mappatura casi d'uso → moduli (dalla slide):

- *Visualizza prodotti* → modulo **Catalog**
- *Aggiungi al carrello* → modulo **Basket**
- *Checkout* → modulo **Ordering**

E — punto interessante — **ogni modulo può seguire internamente una struttura diversa** (N-layer, Clean Architecture, Vertical Slice), esattamente come i microservizi.

Le 6 sfide

1. Richiede forte disciplina architetturale ★

"Questi confini possono facilmente erodersi nel tempo se non vengono gestiti con attenzione."

Questa è **la** sfida centrale: in un monolite modulare **niente ti impedisce fisicamente** di bypassare un'interfaccia. Solo la disciplina del team (e i vincoli di compilazione) tengono in piedi i confini.

2. Rischio di leaky abstraction / accoppiamento stretto

Se l'interfaccia di un modulo non è ben definita, o gli sviluppatori la scavalcano, **i moduli restano strettamente accoppiati** e si perdono i vantaggi della modularità.

3. Risorse condivise

Tutti i moduli tipicamente **condividono lo stesso database** (rischio di conflitti e accoppiamento indesiderato sui dati) e **le stesse risorse di processo** (CPU, memoria, thread pool).

4. Scalabilità non granulare ★

Restando un'unica unità di deployment, **l'intera applicazione scala insieme**. *Non è possibile scalare i singoli moduli in base al loro carico specifico*. Questo limite resterà anche dopo la modularizzazione, ed è ciò che alla fine spingerà ai microservizi.

5. Tempi di build lunghi

Se l'applicazione nel complesso è molto grande, i **full build** restano lenti.

6. Stack tecnologico omogeneo

Tutti i moduli usano **lo stesso stack e gli stessi framework di base**. Niente poliglottismo.

Design: E-Shop Modular Monoliths Architecture

Toolkit aggiornato

Requisiti funzionali: invariati.

Nuovi requisiti non funzionali:

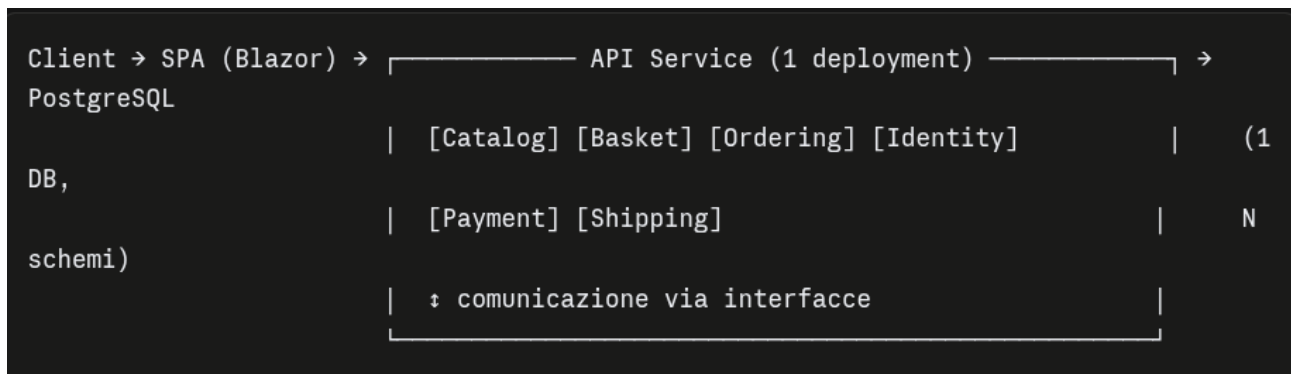
- **Alta coesione interna ai moduli e basso accoppiamento tra i moduli**
- **Migliore autonomia dei team** che lavorano su aree diverse del backend (catalogo, evasione ordini)
- **Migliore manutenibilità e testabilità** dei componenti interni del backend

Restano i requisiti della 3-tier: presentation tier separato, backend API-driven.

Architetture apprese: si aggiunge l'**architettura modulare**, applicata alla **progettazione interna dell'application tier**.

Pattern/principi: si aggiunge la piena consapevolezza dell'anti-pattern **Big Ball of Mud** — *"la nostra progettazione modulare è una contromisura diretta a questo problema"*.

Il design



Cosa NON cambia: la struttura a tre livelli. Il backend **viene ancora distribuito come un singolo blocco**.

Cosa cambia: la **progettazione interna dell'application tier** riflette ora esplicitamente un'architettura modulare.

I moduli progettati per l'e-commerce

Modulo	Responsabilità
Catalog	Informazioni su prodotti, categorie, ricerca
Basket	Creazione del carrello e gestione degli articoli
Ordering	Creazione degli ordini e cronologia
Identity	Autenticazione e autorizzazione utenti
Payment	Integrazione con i gateway di pagamento ed elaborazione pagamenti
Shipping	Gestione delle spedizioni

Ciascun modulo incapsula la **propria logica di business e potenzialmente le proprie preoccupazioni di accesso ai dati**.

Le due regole di isolamento

- Isolamento dei dati:** pur condividendo un unico database relazionale, i moduli **usano schemi diversi o tabelle chiaramente separate** (approfondito nella sezione successiva).
- Isolamento della comunicazione:** la comunicazione tra moduli avviene **tramite interfacce o eventi ben definiti**, per minimizzare accesso diretto, dipendenze dirette e accoppiamento stretto.

Scelte tecnologiche

Tier	Tecnologia	Note
Presentation	Blazor (SPA)	invariato
Application	ASP.NET Core Web API	"la differenza principale risiede nell'organizzazione interna"
Data	PostgreSQL	invariato, ma con schemi separati

Come si implementano tecnicamente i moduli:

"Moduli specifici come catalogo, pagamenti, ecc. vengono implementati come progetti .NET separati o librerie di classi separate all'interno della stessa soluzione."

Perché è importante: questo garantisce una **chiara separazione in fase di compilazione (compile-time)**. Se il modulo Basket non ha un riferimento di progetto al modulo Ordering, il

codice non compila se provi a chiamarlo direttamente. Il compilatore diventa il guardiano dei confini.

Per la comunicazione tra moduli si possono usare:

- **interfacce di servizio chiare**
- **pattern Mediator** per la messaggistica in-process

Best Practice: DB Schema Separation of Modules in Modular Monoliths

Questa sezione merita attenzione perché è la pratica che **rende reale** l'isolamento dei dati.

Definizione e principio

La separazione degli schemi di database è la pratica di isolare i dati di ciascun modulo all'interno del proprio schema, all'interno di un unico database.

Definizione: schema (in PostgreSQL) — un namespace logico dentro un database, che contiene tabelle, viste, funzioni. Permette di avere `catalog.products` e `ordering.orders` nello stesso database, con permessi separati.

Nel nostro caso, dentro il singolo database PostgreSQL:

eshopdb

```
|— catalog (schema) → products, categories
|— basket (schema) → shopping_carts, cart_items
|— ordering (schema) → orders, order_items
```

I 4 motivi per cui è la best practice

1. Isolamento logico dei dati

I dati di ogni modulo vivono in uno schema separato, **indicando chiaramente a quale modulo appartengono**. Aiuta a mantenere integrità e coerenza. Offre un **buon equilibrio tra isolamento e facilità di gestione**.

2. Facilità di gestione

È più semplice da gestire rispetto a **database multipli o meccanismi di persistenza differenti**. Inoltre — punto importante — **le modifiche allo schema di un modulo non impattano gli altri moduli**, il che facilita sviluppo, manutenzione, backup, restore e migrazioni.

3. Performance

Riduce la complessità delle query cross-modulo. Le risorse condivise del database possono essere ottimizzate più efficacemente.

4. Sicurezza

Consente **controllo degli accessi granulare a livello di schema** e semplifica i requisiti di audit e compliance.

Esempio pratico: puoi dare all'utente DB del modulo Catalog i permessi solo su `catalog.*`. Se un bug nel codice del Catalog prova a scrivere su `ordering.orders`, il database stesso lo blocca. L'isolamento smette di dipendere solo dalla disciplina.

Il vantaggio strategico

*"Questo **chiarisce chi possiede i dati** — un passo importante se stai considerando di spostare i moduli su un database dedicato in un'architettura completamente a microservizi. Ci prepara all'evoluzione verso i microservizi."*

È la stessa logica dei moduli: quando estrarrai il modulo Catalog in un microservizio, avrai già **esattamente** l'insieme di tabelle che deve portarsi dietro. Senza schemi separati, dovresti districare un groviglio di join.

Code: E-Shop Modular Monoliths Architecture

Cartella 4-eshop-modular-monoliths-first.

La struttura delle cartelle

eshop-modular-monoliths/

```
|— AppHost/           ← Aspire (orchestrazione)
|— ServiceDefaults/
|
|— Bootstrapper/      ← "assembla" i moduli
|   |— ApiService/    ← progetto API di backend (host dei moduli)
|   |   |— WebApp/     ← SPA, invariata
|   |
|   |— Modules/        ← I MODULI (class library separate)
|       |— Catalog/
|       |— Basket/
|       |— Ordering/
|
|— Shared/            ← libreria condivisa (operazioni comuni, Seed)
```

Definizione: Bootstrapper — il progetto che **assembla e avvia i moduli**. Non contiene logica di business; si limita a registrare i moduli e a comporre l'applicazione. Nel codice reale la struttura è: `ApiService` referencia `Catalog`, `Basket`, `Ordering`; i moduli referenziano `Shared`.

★ **Il punto chiave: l'AppHost NON è cambiato**

Aprendo `AppHost/Program.cs`:

*"La nostra architettura di distribuzione è **esattamente la stessa** della precedente architettura a tre livelli. Abbiamo ancora un database PostgreSQL, un progetto API Service e un progetto WebApp."*

*"Questa è la chiave del concetto di modular monolith: l'unità di deployment del backend non è cambiata. C'è ancora un solo API Service, che è una singola unità di deployment. **La modularità è sviluppata internamente** a quel progetto."*

Questo è il concetto che va fissato: **modularità = cambiamento interno, invisibile dall'esterno.**

Struttura interna di un modulo

Prendendo Modules/Catalog/:

Catalog/

```
|— Data/      → CatalogDbContext, connessione PostgreSQL, data access
| |— Migrations/
| |— Seed/
|— Models/    → entità Product, Category
|— Services/   → ProductService — i CASI D'USO / logica di business
|— Endpoints/  → ProductEndpoints — espone le API del modulo
```

E Modules/Basket/ ha esattamente la stessa struttura: `BasketDbContext`, entità `ShoppingCart` e `CartItem`, `BasketService` con la logica del carrello, `BasketEndpoints` con `get/add/delete` carrello.

Osservazione della lezione: "ogni modulo può gestire tutte le operazioni all'interno della propria class library — logica di business, accesso ai dati, ed esposizione delle API." È un **microservizio in miniatura**, solo che gira nello stesso processo.

Come i moduli vengono registrati

In `ApiService/Program.cs`:

`builder.Services`

```
.AddCatalogModule(builder.Configuration)
.AddBasketModule(builder.Configuration)
.AddOrderingModule(builder.Configuration);
```

```
var app = builder.Build();
```

```
app.UseCatalogModule()
   .UseBasketModule()
   .UseOrderingModule();
```

Ogni modulo espone **due extension method**:

- `AddXxxModule()` → registra i servizi nel container di dependency injection (DbContext, service, repository)
- `UseXxxModule()` → configura middleware e **mappa gli endpoint** del modulo

Il beneficio architetturale: *"questo rende `Program.cs` dell'API Service più pulito e delega le responsabilità ai moduli stessi."* Il bootstrapper non sa nulla di cosa fa il modulo Catalog — sa solo come attivarlo.

Conseguenza chiave: l'endpoint `/products` **non è più definito nell'API Service**, ma **dentro il modulo Catalog**, e viene semplicemente mappato dal bootstrapper. *"La logica di esecuzione per la gestione delle richieste sui prodotti ora risiede nel modulo Catalog."*

Cosa si osserva eseguendo

La dashboard Aspire è identica a prima: WebApp, ApiService, PostgreSQL, pgAdmin.

*"L'aspetto esterno dei nostri servizi non è cambiato, **perché la modularità è una questione interna all'API Service.**"*

Debug del flusso: si mette un **breakpoint in `ProductService.GetProductsAsync()`** dentro il modulo Catalog, si ricarica la pagina prodotti e il breakpoint viene colpito — dimostrando che la richiesta arriva davvero al modulo, passando per `ProductEndpoints`.

Poi il checkout: il metodo `Checkout` del **BasketService** (modulo Basket) elimina il carrello e i suoi articoli, e **crea un record nella tabella orders** (modulo Ordering).

★ **Nota importantissima sul tracing**

*"Durante questo processo i moduli possono comunicare tra loro, ma **la comunicazione interna tra moduli avviene in-process, quindi NON appariranno come nodi separati nel distributed tracing.**"*

Questo è un concetto da tenere a mente per tutto il corso: **la modularità è invisibile al tracing distribuito**, perché non c'è nulla di distribuito. Nel tracing vedrai solo WebApp → ApiService → PostgreSQL. La chiamata BasketService → CatalogService è una semplice chiamata a metodo.

La verifica degli schemi in pgAdmin

Il momento più significativo della sezione. Connettendosi al database:

*"Questa volta **non abbiamo un solo schema.** Abbiamo schemi separati per modulo."*

- Schema catalog → tabella products. Un `SELECT` mostra tutti i prodotti.
- Schema ordering → tabella orders. Un `SELECT` mostra l'ordine appena creato.
- Schema basket → tabelle del carrello.

*"Ogni modulo gestisce le proprie tabelle all'interno del proprio schema. Questa separazione, **anche all'interno di un singolo database**, fornisce un migliore isolamento dei dati tra i moduli."*

Monolith Vertical Considerations (Communication, Transactions, Deployments)

Questa sezione è **strategicamente cruciale** anche se sembra di riepilogo: stabilisce il **baseline** contro cui verranno confrontati i microservizi in tutto il resto del corso.

A) Comunicazione

Poiché **tutte le parti esistono ed eseguono nello stesso processo applicativo sullo stesso server**, la comunicazione è **in-process**.

Definizione: comunicazione in-process — le componenti interagiscono tramite **chiamate dirette a metodi o funzioni** nel codice, senza alcun coinvolgimento della rete.

Esempio della lezione: quando il modulo **Basket** ha bisogno di informazioni su un prodotto, chiama **direttamente un metodo** di un servizio esposto dal modulo **Catalog**.

Aspetto	In un monolite
Velocità	Molto veloce (nanosecondi)
Complessità	Molto facile da implementare
Da gestire	Nessuna latenza di rete, nessuna serializzazione/deserializzazione, nessun retry
Best practice:	anche se le chiamate sono dirette, si comunica sempre tramite interfacce , mai toccando direttamente l'implementazione interna di un altro modulo. È ciò che mantiene i confini.
È possibile anche l'asincrono dentro un monolite: usando code in memoria per attività in background, o integrando un message broker esterno per processi asincroni più robusti su funzionalità specifiche.	

B) Gestione delle transazioni ★

Questo è il **vantaggio più grande del monolite**, ed è quello che si perderà passando ai microservizi.

Con **un singolo database relazionale e un singolo processo**, la gestione delle transazioni è **molto più semplice** che nei microservizi.

Tipicamente una transazione di business:

- interagisce con **una singola istanza di database**
- viene eseguita **nello stesso processo applicativo**

Si possono quindi **sfruttare pienamente le proprietà ACID** del database relazionale, e gli ORM (come **Entity Framework**) offrono meccanismi integrati di gestione delle transazioni, incluso il pattern **Unit of Work**.

Definizione: Unit of Work — pattern che incapsula un insieme di operazioni che devono **avere successo o fallire insieme**, tracciando le modifiche in memoria e scrivendole nel database in un unico commit.

Esempio concreto — il **checkout**, che comporta:

1. creare un record d'ordine
2. aggiornare l'inventario del prodotto

3. elaborare il pagamento (se la logica è interna)
4. inviare la conferma

*"Tutti questi passaggi possono essere incapsulati **all'interno di un'unica transazione di database**. Se un passaggio fallisce, **l'intera transazione può essere annullata (rollback)**, garantendo la coerenza dei dati."*

Il meccanismo: tutte le modifiche sono **tenute in memoria nello stesso transaction scope** e scritte sul database **solo dopo il commit riuscito**. In caso di eccezione, tutto viene scartato.

La frase da ricordare: *"Ottenere questa atomicità è relativamente semplice in un'architettura monolitica. Vedremo in seguito come questo diventi una sfida significativa nei microservizi."*

Questo è esattamente il problema che la **Lezione 12 (transazioni distribuite, pattern Saga)** dovrà risolvere.

C) Deployment

Singola unità di deployment: tutto il codice, inclusi tutti i moduli interni, è impacchettato in **un singolo artefatto** (un JAR/WAR, un set di DLL, **una singola immagine Docker**).

Il lato positivo: per applicazioni piccole o agli inizi, distribuire un singolo artefatto è **più semplice** che distribuire N servizi.

I 5 problemi significativi:

1. **Anche la più piccola modifica** a un modulo richiede **rebuild + retest + redeploy dell'intera applicazione** → deployment poco frequenti, lenti e rischiosi
2. **Un bug in una parte può far crashare l'intero sistema** (tutto condivide lo stesso processo)
3. **Se un deployment fallisce, ne risente l'intera applicazione**
4. Per evitare downtime servono **strategie complesse come il blue-green deployment**
5. **Scaling non granulare:** quando serve capacità, si scala l'intera applicazione, non il singolo modulo sotto carico

*"Immaginate: un grande team di sviluppo che lavora su un'unica codebase, il che porta a un percorso lungo e a volte arduo verso la produzione, che spesso comporta **numerosi test manuali per timore di rompere qualcosa** in un'applicazione grande e complessa."*

Definizione: Blue-Green Deployment — si mantengono due ambienti di produzione identici. Si distribuisce la nuova versione su quello inattivo ("green"), lo si verifica, poi si sposta il traffico. Permette rollback istantaneo, ma raddoppia le risorse.

Problem: Product Page Slowness — Failures During Peak Traffic

Nuovo ciclo. Passo **PROBLEM**.

Lo scenario

L'app diventa popolare: campagna marketing di successo, oppure periodo natalizio. **Aumento improvviso e significativo di utenti.** Molti consultano il catalogo: pagine lista prodotti e pagine dettaglio prodotto.

"Sebbene la nostra architettura sia logicamente solida, questo porta a un aumento del carico su una parte frequentemente visitata dell'app, rivelando colli di bottiglia nelle prestazioni."

La catena della richiesta (e dove si rompe)

Utente → WebApp → chiamata API → ApiService (modulo Catalog) → query SQL → PostgreSQL

In condizioni normali va bene. **Durante i picchi:** centinaia o migliaia di utenti richiedono **ripetutamente le stesse identiche informazioni sui prodotti** in un breve lasso di tempo.

Risultato: **volume elevatissimo di operazioni di lettura sul database.** Il database — e forse l'API stessa — si sovraccaricano nel gestire richieste ripetute. → tempi di risposta più lunghi, rallentamenti, errori.

L'inefficienza fondamentale

Il punto centrale, che è anche la chiave della soluzione:

I dati dei prodotti — nome, descrizione, prezzo, immagine — sono piuttosto statici. È inefficiente interrogare ripetutamente il database per le stesse informazioni.

Stiamo pagando il costo pieno di una query SQL migliaia di volte per ottenere **esattamente lo stesso risultato.**

Sintomi tecnici

- Lentezza percepita sulla pagina lista prodotti e sulla pagina dettaglio
- Generale mancanza di reattività dell'applicazione nei periodi di picco
- Nei dati di monitoraggio: **picco di carico sul database, CPU e operazioni di I/O**
- Nel caso peggiore: **timeout ed errori**

Impatto sul business

Conseguenza	Dettaglio
Esperienza utente scadente	Frustrazione
Abbandono del carrello	<i>"Se le pagine sono troppo lente, gli utenti abbandonano semplicemente i carrelli"</i>
Perdita diretta di vendite e fatturato	Particolarmente grave durante promozioni o alta stagione
Danno reputazionale	<i>"Un sito lento o inaffidabile è la via più rapida per perdere la fiducia dei clienti"</i>
Costi infrastrutturali inefficienti	Se l'unica soluzione è aggiungere hardware al database o raddoppiare l'intero backend solo per gestire le pagine prodotto

La direzione della soluzione

"Dobbiamo velocizzare notevolmente l'accesso a questi dati relativamente statici e frequentemente usati e ridurre il carico sul database PostgreSQL principale, evitando di accedervi per ogni singola richiesta di informazioni sul prodotto."

→ **Caching.**

Learn: Output Caching — Speeding Up Repetitive Requests

Passo **LEARN**.

Definizione

L'output caching è una tecnica di ottimizzazione delle prestazioni in cui l'output completo di una richiesta viene memorizzato in una posizione ad accesso rapido (la cache).

L'output può essere: **una pagina HTML completa, una risposta JSON di un endpoint API**, o qualsiasi risposta la cui generazione sia costosa.

Nota terminologica: l'*output caching* memorizza **il risultato finale già renderizzato**. Si distingue dal *data caching*, che memorizza i dati grezzi (es. l'oggetto Product) lasciando comunque da fare il rendering. L'output caching è più aggressivo e più veloce, ma meno flessibile.

Il meccanismo (3 passi)

1. Prima richiesta (cache miss)

La richiesta segue il percorso normale: query al database, calcoli, rendering del template. L'output viene inviato all'utente **e una copia viene memorizzata nella cache**, associata a una **cache key** (che di solito dipende da URL e parametri della richiesta).

2. Richieste successive (cache hit)

L'applicazione **controlla prima la cache**. Se trova una versione **valida e non scaduta**, la serve direttamente **bypassando tutta l'elaborazione costosa** — query al database e rendering inclusi.

3. Cache miss o scaduta

L'applicazione rigenera il contenuto e **aggiorna la cache** con la nuova versione.

Come risolve il nostro problema specifico

*"Le informazioni sui prodotti — nomi, descrizioni, immagini e persino i prezzi — generalmente **non cambiano ogni secondo o ogni minuto**. Sono contenuti relativamente statici. Questo li rende **ideali per l'output caching**."*

Il flusso:

1. Il primo utente richiede la pagina prodotti → la richiesta va all'API → al database → l'app Blazor renderizza la pagina → **e ne memorizza una copia nell'output cache**

2. Quando **centinaia o migliaia di altri utenti** richiedono la stessa identica pagina poco dopo
→ **servita istantaneamente dalla cache**

Il risultato:

- **Carico sul database drasticamente ridotto** — le query girano solo quando la cache è scaduta o per dati non cacheable
- **Tempi di risposta rapidissimi** per gli utenti
- **Esperienza utente nettamente migliore, soprattutto durante i picchi**

Le 5 considerazioni per implementarlo bene

1. Durata della cache (cache duration / TTL) — è un trade-off:

	Beneficio	Rischio
TTL lungo	Prestazioni migliori, meno carico DB	Servire dati obsoleti (stale)
TTL corto	Dati più aggiornati	Meno cache hit, più carico DB

Per una pagina prodotto: *da pochi minuti a un'ora.*

2. Strategia di invalidazione della cache — *"spesso l'aspetto più complesso della gestione della cache".*

"Cosa succede quando i dati sottostanti cambiano? Se il prezzo di un prodotto viene aggiornato nel database, come ci assicuriamo che la versione in cache venga rimossa o aggiornata?"

Due strategie comuni:

- **Scadenza basata sul tempo (TTL)** — semplice, ma accetti dati stale fino alla scadenza
- **Invalidazione esplicita (eviction)** — quando i dati cambiano, si rimuove attivamente la voce di cache. Più preciso, più complesso.

3. Progettazione della cache key

Serve un modo per **identificare univocamente ogni elemento cacheable**. Si basa tipicamente su **URL della richiesta, query string parameters, o specifici header**.

Esempio: `/products?page=2&category=laptop` e `/products?page=3&category=laptop` devono essere **due chiavi diverse**, altrimenti servi la pagina sbagliata.

4. Posizione della cache (in-memory vs distribuita) ★

	Cache in-memory	Cache distribuita
Dove vive	Nella memoria di ogni singolo server	In un archivio esterno condiviso
Problema	Con più istanze , ogni server ha la sua cache → incoerenze e spreco (lo stesso dato duplicato N volte)	—
Vantaggio	Velocissima, zero infrastruttura	Accessibile da tutte le istanze , coerente
Esempi	—	Redis, Memcached

"Questa soluzione [distribuita] è generalmente preferibile per applicazioni web che possono essere scalate."

Nota come questa considerazione **anticipi già il problema della Lezione 4** (scaling orizzontale): stiamo scegliendo Redis oggi perché sappiamo che domani avremo più istanze.

5. Cosa cacheare

Non tutto è adatto. L'output caching è ideale per dati:

- a cui si accede **frequentemente**
- con **costo di generazione relativamente alto**
- che **non cambiano molto**

Non adatto: contenuti altamente dinamici o **personalizzati** (es. il carrello di un utente specifico, la dashboard personale) — richiederebbero strategie più complesse.

Design: E-Shop Modular Monoliths with Redis Output Caching

Toolkit aggiornato

Nuovi requisiti non funzionali (tutti focalizzati sulle performance):

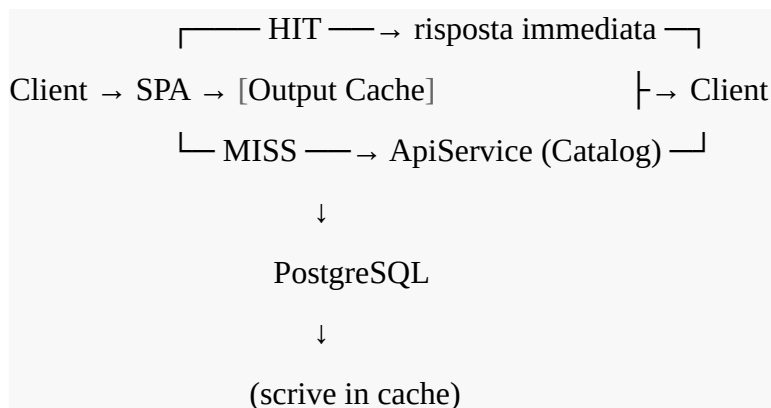
1. **Alte prestazioni e bassa latenza** per le pagine prodotto
2. **Ridotto carico sul database** — spostare le query di lettura ricorrenti fuori da PostgreSQL
3. **Scalabilità per carichi di lavoro read-intensive** — il sistema deve gestire agilmente i picchi

I requisiti precedenti (persistenza, UI separata, backend API-driven) restano.

Nuovi vincoli/assunzioni: bisogna gestire due nuovi processi — **strategia di invalidazione della cache** e **design della cache key** — per mantenere la coerenza tra cache e dati sottostanti.

Nuovo pattern nel toolkit: output caching.

Il design e il flusso della richiesta



Percorso in caso di CACHE HIT:

I dati del prodotto sono già in cache e **non scaduti** → la cache restituisce i dati direttamente alla SPA. **La richiesta non raggiunge mai né l'API Service né PostgreSQL.**

Percorso in caso di CACHE MISS:

1. La richiesta passa all'application tier (backend API)

2. Il **modulo Catalog** elabora la richiesta come prima
3. Interroga PostgreSQL per le informazioni sul prodotto
4. **Prima di restituire la risposta, memorizza i dati nell'output cache** con un tempo di scadenza appropriato
5. I dati vengono restituiti alla SPA
6. Le richieste successive per lo stesso prodotto useranno la cache

Perché una cache distribuita e non in-memory

Il ragionamento esplicito della lezione:

*"Il nostro API Service **potrebbe essere scalato su più istanze in futuro**. Anche con una singola istanza, potremmo eseguirne più di una per bilanciare il carico, e una cache interna al processo di ciascuna istanza **non sarebbe ideale**. Abbiamo quindi bisogno di una cache distribuita: un servizio di caching esterno condiviso a cui tutte le istanze possono accedere."*

Perché Redis — le 5 ragioni

Ragione	Dettaglio
Velocità estrema	Operazioni di lettura e scrittura molto veloci (in-memory data store)
Strutture dati ricche	Supporta stringhe, hash, liste, set, sorted set, ecc.
Diffusione e community	Ampiamente adottato, community eccellente
Opzioni di scalabilità	Supporta il clustering
Ottime librerie client .NET	Facili da usare
Alternative citate: Memcached, NCache.	

Stack finale di questa iterazione

Tier	Tecnologia
Presentation	Blazor WebApp
Cache	Redis
Application	ASP.NET Core Web API (modulare)
Data	PostgreSQL

Code: E-Shop Modular Monoliths with Redis Output Caching

Cartella 5-eshop-modular-monoliths-add-cache.

1. Aggiunta di Redis in AppHost

```
var cache = builder.AddRedis("cache")
    .WithRedisInsight() // UI web per ispezionare la cache
    .WithDataVolume();

builder.AddProject<WebApp>()
```

```
.WithReference(cache) // inietta la connection string Redis
.WithReference(apiService);
```

Cosa fa: aggiunge un **container Redis** all'ambiente distribuito, più **Redis Insight** (l'equivalente di pgAdmin, ma per Redis: un'interfaccia web per ispezionare le chiavi in cache).

Il `WithReference(cache)` sulla `WebApp` le fornisce la **connection string per l'istanza Redis**, necessaria al middleware di output caching.

2. Configurazione nella WebApp

Punto didattico utile della lezione: in ogni progetto ASP.NET Core la configurazione ha **due fasi**:

Fase	Quando	Cosa fa
1. Registrazione servizi	Prima di <code>builder.Build()</code>	Aggiunge servizi al container di dependency injection
2. Pipeline HTTP	Dopo <code>builder.Build()</code>	Configura i middleware che processano le richieste

In `WebApp/Program.cs`:

```
// FASE 1 — prima di Build()
builder.AddRedisOutputCache("cache"); // configura output cache su Redis

var app = builder.Build();

// FASE 2 — dopo Build()
app.UseOutputCache(); // ⚠ IL MIDDLEWARE È OBBLIGATORIO
```

Questo è importante. Dobbiamo aggiungere questo middleware chiamando `app.UseOutputCache()`."

È l'errore più comune: registri il servizio ma dimentichi il middleware, e la cache non funziona mai — senza dare errori.

3. Attivare la cache su una pagina specifica

In `Components/Pages/Products.razor`, **una sola riga**:

```
@attribute [OutputCache(Duration = 5)]
```

"Questo semplice attributo dice ad ASP.NET Core di **memorizzare in cache l'output renderizzato dell'intera pagina Razor per cinque secondi**."

Si possono anche definire **named policies** in `Program.cs` per regole più complesse (variazione per query string, tag per invalidazione, ecc.) e referenziarle qui. Per la demo si usa la configurazione semplice.

Nota pratica: 5 secondi è un valore didattico, scelto per poter vedere hit e miss durante la demo. In produzione per una pagina prodotto useresti minuti.

4. Cosa si osserva eseguendo

La dashboard Aspire ora mostra: **WebApp, ApiService, PostgreSQL, pgAdmin, Redis, Redis Insight.**

Nel resource graph: la **WebApp si connette sia all'ApiService sia alla cache distribuita.**

5. ★ La verifica nel distributed tracing — il risultato numerico

Il momento culminante della lezione. Si azzerano le tracce, si carica la pagina prodotti, si ricarica entro 5 secondi.

PRIMA RICHIESTA (cache miss) — la traccia mostra:

1. HTTP GET verso la pagina prodotti
2. **Tentativo di lettura dalla cache Redis → MISS** (nessun dato per quella chiave)
3. Richiesta HTTP all'endpoint `/products` dell'ApiService
4. L'ApiService esegue **query SQL su PostgreSQL**
5. **Scrittura dell'intera pagina HTML renderizzata nella cache Redis**

SECONDA RICHIESTA entro 5 secondi (cache hit):

1. HTTP GET verso la pagina prodotti
2. **Lettura dalla cache Redis → HIT**
3. Risposta servita. **Nessuna chiamata all'API. Nessuna query SQL.**

I numeri:

Richiesta	Durata
-----------	--------

Prima (cache miss)	~400 ms
--------------------	---------

Seconda (cache hit)	~6,8 ms
---------------------	---------

Circa 60 volte più veloce. Ed è esattamente il punto: il beneficio del caching non è teorico, è misurabile nel tracing.

6. Ispezione con Redis Insight

Aprendo Redis Insight:

- Si vede l'istanza chiamata **cache**
- Aprendola, si vedono **tutte le chiavi presenti**
- Ricaricando la pagina prodotti → **compare la chiave** contenente **l'intera pagina HTML renderizzata**
- **Dopo 5 secondi → la chiave è sparita** (scadenza TTL)

Questo rende visibile e tangibile tutto il meccanismo: chiave, valore, TTL, scadenza.

Problem: Increased Traffic, Can't Handle More Requests w/ Single App Instance

EVAL di questa iterazione → **PROBLEM** della Lezione 4.

Lo scenario

L'applicazione è in produzione e attrae costantemente un elevato numero di utenti simultanei.

Anche con il caching delle pagine prodotto, il volume di richieste al secondo sta sovraccaricando l'API Service.

Perché? Perché il caching ha risolto **solo le letture statiche**. Restano:

- **elaborazione degli ordini**
- **gestione degli account utente**
- **controlli di inventario**
- tutto il **contenuto dinamico e personalizzato** (che non è cacheable)

*"Il caching aiuta per le letture di contenuti statici, ma **il carico combinato di transazioni e richieste di contenuti dinamici sta esaurendo la capacità del nostro singolo API Service.**"*

Il problema — definito con precisione

"Single Instance Bottleneck" (collo di bottiglia a istanza singola)

*"Nonostante la modularità e la cache, il nostro API Service è **ancora distribuito come una singola istanza**. Questa singola istanza ha **risorse limitate**."*

I limiti fisici di una singola istanza:

- numero massimo di **connessioni simultanee**
- quantità finita di **CPU e memoria**
- limite di **banda di rete**
- limite massimo di **richieste al secondo processabili**

*"**Indipendentemente da quanto sia ottimizzato il codice**, non può gestire ulteriori richieste. Esiste un limite preciso a ciò che una singola istanza può fare."*

Questa frase è importante: il problema **non è più risolvibile con ottimizzazioni**. È un limite **architetturale**, e richiede una risposta architetturale.

Sintomi tecnici

- **Aumento generale della latenza** su quasi tutte le funzionalità
- **Utilizzo costantemente elevato di CPU e memoria** sul server che ospita l'API Service
- **Accodamento delle richieste (request queuing)** → tempi di attesa più lunghi
- Sotto carico elevato continuo: **l'istanza può bloccarsi o diventare completamente non responsiva**
- Gli utenti vedono errori **HTTP 503 Service Unavailable**

Impatto sul business

Conseguenza	Dettaglio
UX scadente	L'intero sito appare lento e inaffidabile
Perdita di vendite	Tassi di abbandono del carrello più elevati
Danno reputazionale	<i>"Dà l'impressione che il negozio non sia in grado di gestire il proprio successo"</i>
Impossibilità di	<i>"Se il sito si blocca durante un periodo cruciale di vendite natalizie,</i>

Conseguenza

capitalizzare sui picchi

Dettaglio

la perdita finanziaria è immediata e sostanziale"

La direzione della soluzione

*"Il problema principale è che **la domanda sull'application tier ha superato la capacità di una singola istanza distribuita**. Dobbiamo trovare un modo per **aumentare la capacità di elaborazione complessiva** del nostro livello API, in modo che possa gestire un volume molto maggiore di utenti e richieste simultanee."*

*"Questa necessità fondamentale ci porta direttamente al tema della **scalabilità dell'applicazione**."*

→ **Lezione 4: Scalability — Vertical and Horizontal Scaling**